

Programming Assignment 2

Galactic Logistics Hub

CS 200: Introduction to Programming

Lead TAs: Ahmad Sarfraz & Fatima Nauman

Spring 2026

Assignment Overview

Submission: RollNo_PA2.zip via LMS

Language: C++

Focus: Abstraction, Encapsulation, Inheritance, Polymorphism, Composition, Rule of Three, Static Members, Friend Classes, Function/Constructor Overloading, Linked List, File Handling (read & write)

Read Before Starting

- Read the **entire manual** before writing a single line of code.
- **Skeleton code is provided.** Fill in the function bodies; do not rename files, classes, or members.
- Signatures and output format must match exactly.
- Every class specified here is mandatory, you may not skip any.
- Member names must match the specification exactly (spelling, casing).
- Every function described must be implemented.
- Shallow copies receive **zero credit** for the deep-copy component.
- Memory leaks will result in mark deductions.

Contents

1	Learning Objectives	4
2	System Overview	5
2.1	Ownership Hierarchy	5
2.2	Class Hierarchy	5
3	Mandatory OOP Rules	6
3.1	Encapsulation	6
3.2	Const Correctness	6
3.3	Virtual Destructors	6
3.4	Rule of Three	6
3.5	The <code>this</code> Pointer	6
3.6	Static Members	6
3.7	Friend Class	6
4	Class: Entity (Abstract Base Class)	7
4.1	Attributes	7
4.2	Class Declaration	7
4.3	Function Definitions	7
5	Class Hierarchy 1: Cargo	9
5.1	Abstract Base Class: <code>Cargo</code>	9
5.1.1	Attributes	9
5.1.2	Class Declaration	9
5.1.3	Function Definitions	9
5.2	Derived Class: <code>StandardCargo</code>	11
5.2.1	Attributes	11
5.2.2	Class Declaration	11
5.2.3	Function Definitions	11
5.3	Derived Class: <code>HazardousCargo</code>	12
5.3.1	Attributes	12
5.3.2	Class Declaration	12
5.3.3	Function Definitions	13
5.4	Derived Class: <code>PerishableCargo</code>	14
5.4.1	Attributes	14
5.4.2	Class Declaration	14
5.4.3	Function Definitions	15
6	Class Hierarchy 2: Vessel	16
6.1	Abstract Base Class: <code>Vessel</code>	16
6.1.1	Attributes	16
6.1.2	Class Declaration	16
6.1.3	Deep Copy Requirement	17
6.1.4	Function Definitions	17
6.2	Derived Class: <code>LightFreighter</code>	20
6.2.1	Attributes	20
6.2.2	Class Declaration	20
6.2.3	Function Definitions	20
6.3	Derived Class: <code>HeavyLifter</code>	21
6.3.1	Attributes	21

6.3.2	Class Declaration	21
6.3.3	Function Definitions	22
6.4	Derived Class: StealthRunner	22
6.4.1	Attributes	22
6.4.2	Class Declaration	22
6.4.3	Function Definitions	23
7	The Inspector Class (Friend Class)	24
7.1	Attributes	24
7.2	Class Declaration	24
7.3	Function Definitions	24
8	The CargoLog Struct (Linked List)	26
8.1	Attributes	26
8.2	Struct Declaration	26
8.3	Function Definitions	26
9	The Hub Class	27
9.1	Attributes	27
9.2	Class Declaration	27
9.3	Function Definitions	27
10	Static Members: Summary Reference	31
11	File Format Specification	32
11.1	Input File (fleet.txt)	32
11.1.1	Format Rules	32
11.1.2	Sample Input File	32
11.2	Output Report File (report.txt)	32
11.2.1	Sample Output File	32
12	Required Console Output Format	34
13	Memory Safety Requirements	35
14	File Handling Reference	36
14.1	Required Headers	36
14.2	Reading from a File — std::ifstream	36
14.3	Parsing a Line — std::stringstream	36
14.4	Writing to a File — std::ofstream	37
14.5	Error Handling	37
14.6	Quick Reference — Common Patterns	37
15	File Structure	38
16	Submission Requirements	39
17	Grading Rubric	40
18	Testing Your Code	40
19	Academic Integrity	43

Learning Objectives

Upon completion of this assignment you will have demonstrated proficiency in:

- Designing abstract base classes using pure virtual functions
- Implementing multi-level and hierarchical inheritance
- Applying runtime polymorphism through virtual function dispatch
- Enforcing encapsulation with correct access specifiers
- Using `protected` members in an inheritance chain
- Managing dynamic memory with the Rule of Three
- Using the `this` pointer correctly in assignment operators
- Declaring and using static class members
- Implementing function overloading and constructor overloading
- Granting privileged access via friend classes
- Modelling composition (“has-a” relationships)
- Building, traversing, and freeing a singly-linked list
- Reading structured input files with `ifstream` and `stringstream`
- Writing formatted reports to files with `ofstream`
- Enforcing const correctness throughout a codebase

System Overview

You are building a **Galactic Logistics Hub**: a system that manages a fleet of interstellar cargo vessels. Each vessel carries a manifest of cargo items. The hub can load its fleet from a configuration file, run logistics calculations, and save a report to a file.

Ownership Hierarchy

Hub

```
|-- Vessel** dock      (array of Vessel pointers, OWNED by Hub)
|   |-- Cargo** manifest (array of Cargo pointers, OWNED by each Vessel)
|-- CargoLog* logHead  (singly-linked list of log entries, OWNED by Hub)
```

Ownership means responsibility for deletion:

- The Hub destructor deletes every **Vessel** it holds.
- Each **Vessel** destructor deletes every **Cargo** in its manifest.
- The Hub destructor also frees the entire **CargoLog** linked list.

Class Hierarchy

Entity (abstract)

```
|-- Cargo (abstract)
|   |-- StandardCargo
|   |-- HazardousCargo
|   |-- PerishableCargo
|-- Vessel (abstract)
|   |-- LightFreighter
|   |-- HeavyLifter
|   |-- StealthRunner
```

Inspector (standalone; declared as friend of Vessel)

CargoLog (struct; singly-linked list node used by Hub)

Hub (concrete; manages vessels via composition)

Mandatory OOP Rules

The following rules apply to the **entire** codebase.

Encapsulation

All data members in every class must be **private** (or **protected** where inheritance requires access). Never make attributes **public**.

Const Correctness

Any member function that does not modify the object's state must be declared **const**. This includes all getters, `display()`, `getTotalWeight()`, and `calculateFuel()`.

Virtual Destructors

Every base class (`Entity`, `Cargo`, `Vessel`) must have a **virtual** destructor so that the correct destructor chain fires when a derived object is deleted through a base-class pointer.

Rule of Three

Any class that manages dynamically-allocated memory must implement all three of:

1. Destructor
2. Copy constructor
3. Copy assignment operator

This requirement applies to `Vessel` and `Hub`. Shallow copies receive **zero credit**.

The this Pointer

The copy assignment operator in every applicable class must begin with a self-assignment check:

```
1 if (this == &other) return *this;
```

Static Members

`Cargo` and `Vessel` each maintain a static counter tracking the number of live instances. It is incremented in each constructor and decremented in each destructor. The test driver verifies that these counters return to **0** after all objects go out of scope.

Friend Class

`Inspector` is declared a friend of `Vessel`, granting it direct access to `Vessel`'s private manifest for inspection purposes.

Class: Entity (Abstract Base Class)

OOP Concept: Abstraction

Entity is the root of the inheritance tree. Because it contains a pure virtual function, it is abstract and cannot be instantiated directly.

Attributes

Access	Type	Name
private (static)	int	entityCount
protected	int	id
protected	std::string	name

entityCount is a class-level counter. It is incremented in both the parameterised constructor and the copy constructor, and decremented in the Entity destructor.

Class Declaration

```
1 class Entity {
2 private:
3     static int entityCount;
4
5 protected:
6     int id;
7     std::string name;
8
9 public:
10    Entity(int id, const std::string& name);
11    Entity(const Entity& other);
12    virtual ~Entity();
13    int getID() const;
14    std::string getName() const;
15    static int getEntityCount();
16    virtual void display() const = 0;
17 };
```

Function Definitions

Entity(int id, const std::string& name)

Type: Constructor

Parameters:

- id – A unique integer identifier for this entity.
- name – The display name of this entity.

Returns: N/A

Description: Initialises this->id and this->name with the provided arguments. Increments the static entityCount by 1.

`Entity(const Entity& other)`

Type: Copy Constructor

Parameters:

- `other` – The `Entity` instance to copy from.

Returns: N/A

Description: Initialises `this->id` and `this->name` from `other`. Increments `entityCount` by 1, because a new object is being created.

`virtual ~Entity()`

Type: Virtual Destructor

Parameters: None

Returns: N/A

Description: Decrements `entityCount` by 1. Marked `virtual` to ensure the correct destructor chain is invoked when deleting a derived object through an `Entity*`.

`int getID() const`

Parameters: None

Returns: `int` – the unique id of this entity.

Description: Const getter; does not modify object state.

`std::string getName() const`

Parameters: None

Returns: `std::string` – the `name` of this entity.

Description: Const getter; does not modify object state.

`static int getEntityCount()`

Parameters: None

Returns: `int` – the current value of the static `entityCount`.

Description: Returns the total number of `Entity`-derived objects currently alive. Being `static`, it can be called without an instance as `Entity::getEntityCount()`.

`virtual void display() const = 0`

Parameters: None

Returns: `void`

Description: Pure virtual. Every concrete derived class must override this function and print a human-readable description of the object to `std::cout`. Its presence makes `Entity` an abstract class.

Class Hierarchy 1: Cargo

OOP Concept: Multi-Level Inheritance + Runtime Polymorphism

Cargo inherits from Entity. The three concrete cargo classes inherit from Cargo. This forms a two-level chain from the root. The virtual function `getEffectiveWeight()` is dispatched at runtime, allowing a `Cargo*` to call the correct formula for its actual type.

Abstract Base Class: Cargo

Attributes

Access	Type	Name
private (static)	int	totalCargoCreated
private	std::string	description
private	double	weight

Class Declaration

```
1 class Cargo : public Entity {
2 private:
3     static int totalCargoCreated;
4     std::string description;
5     double weight;
6
7 public:
8     Cargo(int id, const std::string& description, double weight);
9     virtual ~Cargo();
10    std::string getDescription() const;
11    double getWeight() const;
12    static int getTotalCargoCreated();
13    void setTotalCargoCreated(int newTotalCargoCreated);
14    virtual double getEffectiveWeight() const = 0;
15    virtual Cargo* clone() = 0;
16    virtual void display() const override;
17 };
```

Important Note

`Cargo::display()` is **not** pure virtual here, it provides a default implementation that calls `getEffectiveWeight()` (which IS pure virtual). Derived classes may override `display()` to append extra type-specific information. The class remains abstract because of `getEffectiveWeight()` and `clone()`.

Function Definitions

```
Cargo(int id, const std::string& description, double weight)
```

Type: Constructor

Parameters:

- `id` – Unique integer ID; forwarded to `Entity(id, description)`.
- `description` – Human-readable name of the cargo item (also used as the Entity name).

- **weight** – Raw physical weight in kg. Must be > 0.

Returns: N/A

Description: Calls `Entity(id, description)` in the initialiser list. Sets `this->description` and `this->weight`. Increments `totalCargoCreated` by 1.

`virtual ~Cargo()`

Type: Virtual Destructor

Parameters: None

Returns: N/A

Description: Decrements `totalCargoCreated` by 1. The body may be otherwise empty. `virtual` ensures correct dispatch.

`std::string getDescription() const`

Parameters: None

Returns: `std::string` – the cargo description.

Description: Const getter; does not modify object state.

`double getWeight() const`

Parameters: None

Returns: `double` – the raw physical weight.

Description: Returns the raw weight, not the surcharge-adjusted effective weight. Const getter.

`static int getTotalCargoCreated()`

Parameters: None

Returns: `int` – the current value of `totalCargoCreated`.

Description: Returns how many `Cargo`-derived objects are currently alive (constructed but not yet destructed).

`static void setTotalCargoCreated(int newTotalCargoCreated)`

Type: Static Member Function

Parameters: `newTotalCargoCreated` – New value to assign to the static counter.

Returns: N/A

Description: Sets the value of the static variable `totalCargoCreated`. This function should generally only be used for testing or controlled internal updates.

`virtual double getEffectiveWeight() const = 0`

Parameters: None

Returns: `double`

Description: Pure virtual. Each derived class computes its own effective (logistical) weight as described in the relevant subsection below. `Vessel::getTotalWeight()` calls this function through a base-class pointer – this is runtime polymorphism in action.

```
virtual Cargo* clone() = 0
```

Parameters: None

Returns: Cargo* – pointer to a copy of *this.

Description: Pure virtual. Each derived class increments totalCargoCreated and returns new DerivedClass(*this).

```
virtual void display() const override
```

Parameters: None

Returns: void

Description: Prints the cargo description, raw weight, and effective weight to std::cout. Uses std::fixed and std::setprecision(2). Exact format:

```
[Cargo] FoodSupplies  Raw: 10.00 kg  Effective: 10.00 kg
```

Leading two spaces are required. Derived classes may override this to append type-specific information on the same line.

Derived Class: StandardCargo

OOP Concept: Hierarchical Inheritance

StandardCargo, HazardousCargo, and PerishableCargo all derive from the single base class Cargo. This is *hierarchical inheritance* — multiple classes sharing one common base.

Effective weight formula:

$$\text{effectiveWeight} = \text{weight}$$

Attributes

Access	Type	Name
--------	------	------

<i>No additional attributes — all data members are inherited from Cargo.</i>		
--	--	--

Class Declaration

```
1 class StandardCargo : public Cargo {
2 public:
3     StandardCargo(int id, const std::string& description, double weight)
4         ;
5     double getEffectiveWeight() const override;
6     Cargo* clone() override;
7     void display() const override;
8 };
```

Function Definitions

```
StandardCargo(int id, const std::string& description, double weight)
```

Type: Constructor

Parameters:

- id – Unique integer ID, forwarded to Cargo.

- **description** – Cargo description.
- **weight** – Physical weight in kg.

Returns: N/A

Description: Calls `Cargo(id, description, weight)` in the initialiser list. No additional data members to initialise.

`double getEffectiveWeight() const override`

Parameters: None

Returns: double

Description: Returns `getWeight()` unchanged. Standard cargo carries no weight surcharge.

`Cargo* clone() override`

Parameters: None

Returns: `Cargo*` – a new copy of `*this`.

Description: Follows standard clone protocol.

`void display() const override`

Parameters: None

Returns: void

Description: Calls `Cargo::display()` and appends the string " [Type: Standard]" on the same line.

Derived Class: HazardousCargo

OOP Concept: Constructor Overloading

`HazardousCargo` provides **two** constructors with different parameter lists – this is constructor overloading. Both construct a valid object; the first uses a default hazard level while the second accepts a custom one.

Effective weight formula:

$$\text{effectiveWeight} = \text{weight} \times 1.2$$

The `hazardLevel` integer (1–5) is a descriptive rating. It does **not** affect the effective weight; it is only displayed by `display()`.

Attributes

Access	Type	Name
private	int	hazardLevel

Class Declaration

```
1 class HazardousCargo : public Cargo {
2 private:
3     int hazardLevel;
4 }
```

```

5 public:
6     // Constructor overload 1: hazardLevel defaults to 1
7     HazardousCargo(int id, const std::string& description, double weight
8         );
9
10    // Constructor overload 2: custom hazardLevel
11    HazardousCargo(int id, const std::string& description,
12        double weight, int hazardLevel);
13
14    int getHazardLevel() const;
15    double getEffectiveWeight() const override;
16    Cargo* clone() override;
17    void display() const override;
18 };

```

Function Definitions

HazardousCargo(int id, const std::string& description, double weight)

Type: Constructor — overload 1

Parameters:

- **id** – Unique integer ID.
- **description** – Cargo description.
- **weight** – Physical weight in kg.

Returns: N/A

Description: Calls `Cargo(id, description, weight)`. Sets `hazardLevel = 1` (default).

HazardousCargo(int id, const std::string& description, double weight, int hazardLevel)

Type: Constructor — overload 2

Parameters:

- **id** – Unique integer ID.
- **description** – Cargo description.
- **weight** – Physical weight in kg.
- **hazardLevel** – Integer in range [1, 5]. If the provided value is outside this range, clamp it silently to the nearest boundary.

Returns: N/A

Description: Calls `Cargo(id, description, weight)`. Sets `this->hazardLevel` to the (possibly clamped) value.

int getHazardLevel() const

Parameters: None

Returns: `int` – the hazard level of this cargo.

Description: Const getter.

double getEffectiveWeight() const override

Parameters: None

Returns: `double`

Description: Returns `getWeight() * 1.2`. The 20% surcharge accounts for additional containment and handling required for hazardous materials.

Cargo* clone() override

Parameters: None

Returns: Cargo* – a copy.

Description: Follows standard clone protocol.

void display() const override

Parameters: None

Returns: void

Description: Calls `Cargo::display()` and appends " [Type: Hazardous | Level: X]" on the same line, where X is `hazardLevel`.

Derived Class: PerishableCargo

Effective weight formula:

$$\text{effectiveWeight} = \text{weight} + 5.0$$

The flat 5 kg surcharge accounts for the weight of refrigeration equipment. `expiryDays` stores the shelf life in days and is printed in `display()`.

Attributes

Access	Type	Name
private	int	expiryDays

Class Declaration

```
1 class PerishableCargo : public Cargo {
2 private:
3     int expiryDays;
4
5 public:
6     PerishableCargo(int id, const std::string& description,
7                     double weight, int expiryDays);
8     int getExpiryDays() const;
9     double getEffectiveWeight() const override;
10    Cargo* clone() override;
11    void display() const override;
12};
```

Function Definitions

```
PerishableCargo(int id, const std::string& description, double weight, int expiryDays)
```

Type: Constructor

Parameters:

- `id` – Unique integer ID.
- `description` – Cargo description.
- `weight` – Physical weight in kg.
- `expiryDays` – Shelf life in days (> 0). If ≤ 0 , print a warning to `stderr` and set it to 1.

Returns: N/A

Description: Calls `Cargo(id, description, weight)` and sets `this->expiryDays` (after validation). **Note:** To print to `stderr` in C++, use:

```
std::cerr << "Invalid expiryDays. Defaulting to 1.\n";
```

```
int getExpiryDays() const
```

Parameters: None

Returns: `int`

Description: Const getter. Returns `expiryDays`.

```
double getEffectiveWeight() const override
```

Parameters: None

Returns: `double`

Description: Returns `getWeight() + 5.0`.

```
Cargo* clone() override
```

Parameters: None

Returns: `Cargo*`

Description: Follows standard clone protocol.

```
void display() const override
```

Parameters: None

Returns: `void`

Description: Calls `Cargo::display()` and appends " [Type: Perishable | Expires in: X days]" on the same line, where X is `expiryDays`.

Class Hierarchy 2: Vessel

OOP Concept: Rule of Three + Static Members + Friend Class + Overloading

Vessel inherits from Entity. As a class that owns a dynamically-allocated array, it requires the Rule of Three. It also carries a static member counter and grants Inspector friend access to its private manifest.

Abstract Base Class: Vessel

Attributes

Access	Type	Name
private (static)	int	totalVesselsCreated
private	Cargo**	manifest
private	int	currentLoad
private	int	maxCapacity

Class Declaration

```
1 class Vessel : public Entity {
2 private:
3     static int totalVesselsCreated;
4     Cargo** manifest;
5     int currentLoad;
6     int maxCapacity;
7
8     friend class Inspector; // Inspector may access private members
9                               directly
10 public:
11     // Constructor overload 1: maxCapacity defaults to 10
12     Vessel(int id, const std::string& name);
13
14     // Constructor overload 2: custom capacity
15     Vessel(int id, const std::string& name, int maxCapacity);
16
17     Vessel(const Vessel& other);
18     Vessel& operator=(const Vessel& other);
19     virtual ~Vessel();
20
21     // Function overload 1: append cargo
22     void addCargo(Cargo* item);
23
24     // Function overload 2: place cargo at a specific slot
25     bool addCargo(Cargo* item, int slot);
26
27     double getTotalWeight() const;
28     int getCurrentLoad() const;
29     int getMaxCapacity() const;
30     static int getTotalVesselsCreated();
31
32     virtual double calculateFuel(int distance) const = 0;
33     virtual Vessel* clone() const = 0;
34     virtual void display() const override = 0;
```



```

35     virtual void writeReport(std::ofstream& out) const = 0;
36 };

```

Important Note

Constructor overloading: Both `Vessel(int, string)` and `Vessel(int, string, int)` are valid; the first defaults `maxCapacity` to 10.

Function overloading: `addCargo` has two forms. The one-argument version appends in order; the two-argument version places cargo at a specific index slot.

Virtual `clone()`: Required so that `Hub`'s copy constructor can polymorphically deep-copy each vessel. Implement in every derived vessel class as `return new DerivedClass(*this);`.

Deep Copy Requirement

The copy constructor and assignment operator must:

1. Allocate a new `Cargo*` array of size `maxCapacity`.
2. For each $i \in [0, \text{currentLoad})$, set `manifest[i] = other.manifest[i]->clone()`.
3. Initialise remaining slots to `nullptr`.
4. Never copy raw pointers directly.

Function Definitions

```
Vessel(int id, const std::string& name)
```

Type: Constructor — overload 1

Parameters:

- `id` – Unique integer ID.
- `name` – Vessel name.

Returns: N/A

Description: Calls `Entity(id, name)`. Sets `maxCapacity = 10`, `currentLoad = 0`. Allocates `manifest = new Cargo*[10]` and initialises all entries to `nullptr`. Increments `totalVesselsCreated`.

```
Vessel(int id, const std::string& name, int maxCapacity)
```

Type: Constructor — overload 2

Parameters:

- `id` – Unique integer ID.
- `name` – Vessel name.
- `maxCapacity` – Maximum cargo slots. Must be > 0 .

Returns: N/A

Description: Calls `Entity(id, name)`. Sets `this->maxCapacity`, `currentLoad = 0`. Allocates `manifest = new Cargo*[maxCapacity]`, initialises entries to `nullptr`. Increments `totalVesselsCreated`.

```
Vessel(const Vessel& other)
```

Type: Copy Constructor

Parameters:

- **other** – The vessel to copy from.

Returns: N/A

Description: Chains to the `Entity` copy constructor. Copies `currentLoad` and `maxCapacity`. Allocates a fresh `manifest` array. For each i in $[0, \text{currentLoad})$, calls `other.manifest[i] -> clone()` and stores the result. Sets remaining slots to `nullptr`. Increments `totalVesselsCreated`.

`Vessel& operator=(const Vessel& other)`

Type: Copy Assignment Operator

Parameters:

- **other** – the source vessel.

Returns: `Vessel&` – reference to `*this`.

Description:

1. **Self-assignment guard:** if `(this == &other)` return `*this`; — uses the `this` pointer.
2. Delete every non-null `Cargo*` in the existing `manifest`, then `delete[] manifest`.
3. Copy `currentLoad` and `maxCapacity` from `other`.
4. Allocate a new `manifest`, clone each cargo item.
5. Return `*this`.

`virtual ~Vessel()`

Type: Virtual Destructor

Parameters: None

Returns: N/A

Description: Iterates over `manifest[0..currentLoad-1]`, deletes every non-null `Cargo*`, then calls `delete[] manifest`. Decrements `totalVesselsCreated`.

`void addCargo(Cargo* item)`

Type: Member Function — overload 1

Parameters:

- **item** – Pointer to a heap-allocated `Cargo` object. The vessel takes ownership; do not delete it externally after calling this function.

Returns: `void`

Description: If `currentLoad == maxCapacity`, prints "Cargo Capacity Reached!" to `std::cout` and returns without modifying the `manifest`. Otherwise, stores `item` at `manifest[currentLoad]` and increments `currentLoad`.

`bool addCargo(Cargo* item, int slot)`

Type: Member Function — overload 2

Parameters:

- **item** – Pointer to a heap-allocated `Cargo` object.
- **slot** – Zero-based index at which to place the item.

Returns: `bool` – `true` on success, `false` on failure.

Description: Validates that `slot` is in $[0, \text{maxCapacity})$ and that `manifest[slot] ==`

`nullptr`. If either condition fails, prints "Invalid slot or slot already occupied." and returns `false`. Otherwise places `item` at `manifest[slot]`, updates `currentLoad` if necessary, and returns `true`. *Note: this overload exists to demonstrate function overloading. It is not called by the file-loading path.*

`double getTotalWeight() const`

Parameters: None

Returns: `double` – sum of all effective weights.

Description: Iterates over `manifest[0..currentLoad-1]`; for each non-null pointer, adds `getEffectiveWeight()` to an accumulator and returns the total. Returns 0.0 for an empty vessel. This call demonstrates runtime polymorphism: the virtual dispatch selects the correct formula for each concrete cargo type.

`int getCurrentLoad() const`

Parameters: None

Returns: `int` – current number of cargo items.

Description: Const getter.

`int getMaxCapacity() const`

Parameters: None

Returns: `int` – maximum number of cargo slots.

Description: Const getter.

`static int getTotalVesselsCreated()`

Parameters: None

Returns: `int`

Description: Returns the number of currently live `Vessel` derived objects.

`virtual double calculateFuel(int distance) const = 0`

Parameters:

- `distance` – Distance in light-years.

Returns: `double` – fuel units required.

Description: Pure virtual. Each derived vessel type implements its own fuel formula as specified in the subsections below.

`virtual Vessel* clone() const = 0`

Parameters: None

Returns: `Vessel*` – heap-allocated deep copy.

Description: Pure virtual. Each derived class returns `new DerivedClass(*this);`. Used by Hub's copy constructor and assignment operator to deep-copy each vessel polymorphically.

```
virtual void display() const override = 0
```

Parameters: None

Returns: void

Description: Pure virtual. Derived classes print vessel name, type, cargo count, total effective weight, and fuel for 100 ly to `std::cout`.

```
virtual void writeReport(std::ofstream& out) const = 0
```

Parameters:

- `out` – An open `ofstream` to write to.

Returns: void

Description: Pure virtual. Identical output to `display()` but directed to `out` instead of `std::cout`. Each derived class implements this by repeating the same formatting logic using `out` instead of `cout`.

Derived Class: LightFreighter

Fuel formula:

$$\text{fuel} = \text{distance} \times (\text{totalWeight} \times 0.5)$$

Attributes

Access	Type	Name
--------	------	------

No additional attributes — all data members are inherited from `Vessel`.

Class Declaration

```
1 class LightFreighter : public Vessel {
2 public:
3     LightFreighter(int id, const std::string& name);
4     LightFreighter(int id, const std::string& name, int maxCapacity);
5     double calculateFuel(int distance) const override;
6     Vessel* clone() const override;
7     void display() const override;
8     void writeReport(std::ofstream& out) const override;
9 };
```

Function Definitions

```
LightFreighter(int id, const std::string& name)
```

Type: Constructor — overload 1

Description: Calls `Vessel(id, name)`. No additional members.

```
LightFreighter(int id, const std::string& name, int maxCapacity)
```

Type: Constructor — overload 2

Description: Calls `Vessel(id, name, maxCapacity)`.

```
double calculateFuel(int distance) const override
```

Parameters: distance – distance in ly.

Returns: double

Description: Returns $\text{distance} \times (\text{getTotalWeight}() \times 0.5)$.

```
Vessel* clone() const override
```

Returns: Vessel*

Description: Returns `new LightFreighter(*this);`.

```
void display() const override
```

Parameters: None

Returns: void

Description: Prints the following block to `std::cout` using `std::fixed` and `std::setprecision(2)`:

```
Vessel: Firefly [Type: LightFreighter] Cargo Items: 3 / 10
Total Effective Weight: 23.00 kg
Fuel Required (100 ly): 1150.00 units
-----
```

Calls `calculateFuel(100)` for the fuel line. The separator line must be exactly 42 dashes.

```
void writeReport(std::ofstream& out) const override
```

Parameters: out – open output file stream.

Returns: void

Description: Identical output to `display()` but directed to out.

Derived Class: HeavyLifter

Fuel formula:

$$\text{fuel} = (\text{distance} \times \text{totalWeight} \times 1.5) + 500$$

Attributes

Access	Type	Name
--------	------	------

No additional attributes — all data members are inherited from Vessel.

Class Declaration

```
1 class HeavyLifter : public Vessel {
2 public:
3     HeavyLifter(int id, const std::string& name);
4     HeavyLifter(int id, const std::string& name, int maxCapacity);
5     double calculateFuel(int distance) const override;
6     Vessel* clone() const override;
7     void display() const override;
8     void writeReport(std::ofstream& out) const override;
9 };
```

Function Definitions

```
HeavyLifter(int id, const std::string& name)
```

Description: Calls `Vessel(id, name)`.

```
HeavyLifter(int id, const std::string& name, int maxCapacity)
```

Description: Calls `Vessel(id, name, maxCapacity)`.

```
double calculateFuel(int distance) const override
```

Parameters: `distance` – distance in ly.

Returns: `double`

Description: Returns $(\text{distance} \times \text{getTotalWeight}() \times 1.5) + 500$.

```
Vessel* clone() const override
```

Description: Returns `new HeavyLifter(*this);`.

```
void display() const override
```

Description: Same format as `LightFreighter::display()` but with type label `[Type: HeavyLifter]`.

```
void writeReport(std::ofstream& out) const override
```

Description: Same as `display()` directed to `out`.

Derived Class: StealthRunner

OOP Concept: Extra Attribute in Derived Class

`StealthRunner` adds a `cloakRating` (0.0–1.0) that acts as a fuel multiplier, rewarding lower-signature designs with reduced consumption.

Fuel formula:

$$\text{fuel} = \text{distance} \times \text{totalWeight} \times \text{cloakRating}$$

Attributes

Access	Type	Name
private	double	cloakRating

Class Declaration

```
1 class StealthRunner : public Vessel {
2 private:
3     double cloakRating;
4
5 public:
6     StealthRunner(int id, const std::string& name, double cloakRating);
7     StealthRunner(int id, const std::string& name,
```

```

8         int maxCapacity, double cloakRating);
9     double getCloakRating() const;
10    double calculateFuel(int distance) const override;
11    Vessel* clone() const override;
12    void display() const override;
13    void writeReport(std::ofstream& out) const override;
14 };

```

Function Definitions

StealthRunner(int id, const std::string& name, double cloakRating)

Type: Constructor — overload 1

Parameters:

- id, name – forwarded to Vessel(id, name).
- cloakRating – value in [0.0, 1.0]; clamped silently if out of range.

Description: Calls Vessel(id, name). Sets this->cloakRating (clamped).

StealthRunner(int id, const std::string& name, int maxCapacity, double cloakRating)

Type: Constructor — overload 2

Description: Calls Vessel(id, name, maxCapacity). Sets this->cloakRating (clamped).

double getCloakRating() const

Returns: double

Description: Const getter. Returns cloakRating.

double calculateFuel(int distance) const override

Returns: double

Description: Returns distance × getTotalWeight() × cloakRating.

Vessel* clone() const override

Description: Returns new StealthRunner(*this);.

void display() const override

Description: Same layout as other vessel types but with type label [Type: StealthRunner | CloakRating: X.XX].

void writeReport(std::ofstream& out) const override

Description: Same as display() directed to out.

The Inspector Class (Friend Class)

OOP Concept: Friend Classes

Declaring `Inspector` as a friend of `Vessel` grants `Inspector`'s member functions direct access to `Vessel`'s private attributes (`manifest`, `currentLoad`, `maxCapacity`) without any public getters.

Important Note

Friendship is not inherited and not reciprocal. `Inspector` being a friend of `Vessel` does **not** make it a friend of `LightFreighter`, `HeavyLifter`, or `StealthRunner`, nor does it make `Vessel` a friend of `Inspector`.

Attributes

Access	Type	Name
<i>No data members — <code>Inspector</code> is a stateless utility class.</i>		

Class Declaration

```
1 class Inspector {
2 public:
3     void inspect(const Vessel& vessel) const;
4     int  countNullSlots(const Vessel& vessel) const;
5 };
```

Function Definitions

```
void inspect(const Vessel& vessel) const
```

Parameters:

- `vessel` – The vessel to inspect (const reference).

Returns: void

Description: Accesses `vessel.manifest`, `vessel.currentLoad`, and `vessel.maxCapacity` **directly** (possible because `Inspector` is a friend of `Vessel`). Prints the inspection report shown below. Slots beyond `currentLoad` are marked (empty).

```
=== Inspector Report: Firefly ===
Capacity: 3 / 10
Manifest:
  [0] FoodSupplies   | Effective: 10.00 kg
  [1] Uranium        | Effective: 6.00 kg
  [2] Vaccines       | Effective: 7.00 kg
  [3] (empty)
  ...
=====
```



```
int countNullSlots(const Vessel& vessel) const
```

Parameters:

- `vessel` – The vessel to inspect.

Returns: `int` – number of `nullptr` entries in the manifest.

Description: Iterates over `vessel.manifest[0..maxCapacity-1]` and counts `nullptr` entries. Direct private member access is only possible due to the friend declaration.

The CargoLog Struct (Linked List)

OOP Concept: Singly-Linked List + Dynamic Memory

CargoLog is a simple singly-linked list used by the Hub to record a chronological log of every cargo item successfully loaded from the input file.

Required operations: **append at tail** (in `Hub::appendLog`) and **full traversal + deletion** (in Hub's destructor and `clearLog()`). No sorting or mid-list deletion is required.

Attributes

Access	Type	Name
public	std::string	cargoDescription
public	std::string	vesselName
public	CargoLog*	next

As a *struct*, all members are *public* by default. *next* is always initialised to *nullptr* by the constructor.

Struct Declaration

```
1 struct CargoLog {
2     std::string cargoDescription;
3     std::string vesselName;
4     CargoLog*   next;
5
6     CargoLog(const std::string& desc, const std::string& vessel);
7 };
```

Function Definitions

CargoLog(const std::string& desc, const std::string& vessel)

Type: Constructor

Parameters:

- **desc** – Description of the cargo item.
- **vessel** – Name of the vessel it was loaded onto.

Returns: N/A

Description: Sets `cargoDescription = desc`, `vesselName = vessel`, and `next = nullptr`.

The Hub Class

OOP Concept: Composition

Hub does not inherit from Entity. It **owns** its vessels (composition) and manages a CargoLog linked list. Its lifetime governs the lifetime of all contained objects.

Attributes

Access	Type	Name
private	Vessel**	dock
private	int	numVessels
private	int	dockCapacity
private	CargoLog*	logHead

Class Declaration

```
1 class Hub {
2 private:
3     Vessel**    dock;
4     int         numVessels;
5     int         dockCapacity;
6     CargoLog*   logHead;
7
8     void appendLog(const std::string& desc, const std::string& vessel);
9     void clearLog();
10
11 public:
12     Hub();
13     Hub(const Hub& other);
14     Hub& operator=(const Hub& other);
15     ~Hub();
16
17     void addVessel(Vessel* v);
18     Vessel* findVessel(const std::string& name) const; // overload 1
19     Vessel* findVessel(int index) const; // overload 2
20
21     void loadFleet(const char* filename);
22     void processLogistics() const;
23     void printLog() const;
24     void saveReport(const char* filename) const;
25 };
```

Function Definitions

Hub()

Type: Default Constructor

Parameters: None

Returns: N/A

Description: Sets dockCapacity = 20. Allocates dock = new Vessel*[20] and initialises all entries to nullptr. Sets numVessels = 0 and logHead = nullptr.

`Hub(const Hub& other)`

Type: Copy Constructor

Parameters:

- `other` – Source Hub to copy from.

Returns: N/A

Description: Copies `numVessels` and `dockCapacity`. Allocates a new `dock` array. For each vessel in `other.dock[0..numVessels-1]`, calls `other.dock[i]->clone()` and stores the result. Copies the `CargoLog` linked list by traversing `other.logHead` and calling `appendLog()` for each node.

`Hub& operator=(const Hub& other)`

Type: Copy Assignment Operator

Parameters:

- `other` – Source Hub.

Returns: `Hub&`

Description:

1. Self-assignment guard: `if (this == &other) return *this;.`
2. Delete all vessels in `dock`; call `delete[] dock`.
3. Call `clearLog()` to free the existing linked list.
4. Deep-copy vessels and log from `other` (same logic as the copy constructor).
5. Return `*this`.

`~Hub()`

Type: Destructor

Parameters: None

Returns: N/A

Description: Deletes each non-null `Vessel*` in `dock[0..numVessels-1]`. Calls `delete[] dock`. Calls `clearLog()` to free the entire linked list.

`void addVessel(Vessel* v)`

Parameters:

- `v` – Pointer to a heap-allocated vessel. The Hub takes ownership.

Returns: `void`

Description: If `numVessels == dockCapacity`, prints "Dock is full. Cannot add more vessels." and returns without modifying the dock. Otherwise, stores `v` at `dock[numVessels]` and increments `numVessels`.

`Vessel* findVessel(const std::string& name) const`

Type: Member Function — overload 1 (function overloading demo)

Parameters:

- `name` – Vessel name to search for (case-sensitive).

Returns: `Vessel*` – pointer to the matching vessel, or `nullptr` if not found.

Description: Iterates over `dock[0..numVessels-1]` and returns the first vessel whose `getName()` equals `name`.

```
Vessel* findVessel(int index) const
```

Type: Member Function — overload 2 (function overloading demo)

Parameters:

- `index` – Zero-based index into the dock array.

Returns: `Vessel*` – `dock[index]`, or `nullptr` if the index is out of range.

Description: Returns `dock[index]` if $0 \leq \text{index} < \text{numVessels}$; otherwise returns `nullptr`.

```
void loadFleet(const char* filename)
```

Parameters:

- `filename` – Path to the input fleet configuration file.

Returns: `void`

Description: Opens the file; if it cannot be opened, prints "Error opening file: <filename>" and returns. Parses the file according to the format in Section 11. Creates the appropriate derived `Vessel` subclass, populates its manifest with `Cargo` objects, and calls `addVessel()`. For every cargo item successfully added to a vessel, calls `appendLog(cargoDescription, vesselName)`.

Error handling within parsing:

- **Unknown vessel type** – prints "Unknown vessel type" and skips the entire vessel block (including all of its cargo lines up to the next END).
- **Unknown cargo type** – prints "Unknown cargo type" and skips that cargo line; the vessel is still added.
- **Negative weight** – skips that cargo line silently; the vessel is still added.

```
void processLogistics() const
```

Parameters: None

Returns: `void`

Description: Iterates over `dock[0..numVessels-1]` and calls `display()` on each vessel through the base-class pointer. This is the primary demonstration of runtime polymorphism: the correct `display()` for each vessel type is selected automatically through the virtual dispatch mechanism.

```
void printLog() const
```

Parameters: None

Returns: `void`

Description: Traverses the `CargoLog` linked list from `logHead` to the last node, printing each entry in the format:

```
[LOG] FoodSupplies loaded onto Firefly
```

If the list is empty, prints "No cargo log entries.".

```
void saveReport(const char* filename) const
```

Parameters:

- `filename` – Path to the output report file (created or overwritten).

Returns: `void`

Description: Opens an `ofstream` to `filename`. If it fails, prints "Error writing report." to `stderr` and returns. Writes a header line, then calls `writeReport(out)` on each vessel in the dock. Appends the full cargo log at the end. Closes the stream. See Section 14 for a C++ file handling reference.

```
void appendLog(const std::string& desc, const std::string& vessel) [private]
```

Parameters:

- `desc` – Cargo description.
- `vessel` – Vessel name.

Returns: `void`

Description: Allocates a new `CargoLog` node and appends it to the **tail** of the linked list. If `logHead == nullptr`, sets `logHead` to the new node. A tail pointer (or traversal to the end) must be used to implement efficient appending.

```
void clearLog() [private]
```

Parameters: None

Returns: `void`

Description: Traverses the linked list from `logHead`, deletes each node, and sets `logHead = nullptr` when done. Called from the destructor and assignment operator to release all log memory.

Static Members: Summary Reference

OOP Concept: Static Members

A static data member belongs to the class, not to any particular object. It is shared across all instances. Static member functions can be called without an instance using the class-name syntax.

Class	Member	Type	Behaviour
Entity	entityCount	int	+1 ctor, -1 dtor
Cargo	totalCargoCreated	int	+1 ctor, -1 dtor
Vessel	totalVesselsCreated	int	+1 ctor, -1 dtor

Static members must be defined in exactly **one** .cpp file (not in the header):

```
1 // Entity.cpp
2 int Entity::entityCount = 0;
3
4 // Cargo.cpp
5 int Cargo::totalCargoCreated = 0;
6
7 // Vessel.cpp
8 int Vessel::totalVesselsCreated = 0;
```

All of these must read **0** once every object has been destroyed.

File Format Specification

Input File (fleet.txt)

Format Rules

1. Each vessel block starts with a *header line*: `<VesselType> <VesselID> <VesselName>`
 - `VesselType`: `LightFreighter`, `HeavyLifter`, or `StealthRunner`.
 - `VesselID`: integer.
 - `VesselName`: single word (no spaces).
2. For `StealthRunner`, the header line has an extra token: `StealthRunner <ID> <Name> <CloakRating>`
3. Each header is followed by zero or more cargo lines, terminated by the single word `END` on its own line.
4. Cargo line format: `<CargoType> <CargoID> <Description> <Weight>`
 - `CargoType`: `Standard`, `Hazardous`, or `Perishable`.
 - `CargoID`: integer.
 - `Description`: single word.
 - `Weight`: non-negative double.
5. For `Hazardous`: append `<HazardLevel>` (int) after the weight.
6. For `Perishable`: append `<ExpiryDays>` (int) after the weight.
7. Blank lines are permitted anywhere and must be silently skipped.

Sample Input File

```
LightFreighter 1 Firefly
Standard      101 FoodSupplies  10.0
Hazardous     102 Uranium        5.0  3
Perishable    103 Vaccines       2.0  7
END
```

```
HeavyLifter 2 Goliath
Standard 201 MachinePartsA 40.0
Standard 202 MachinePartsB 35.0
Hazardous 203 Acids 8.0
END
```

```
StealthRunner 3 Phantom 0.75
Perishable 301 BioSamples 1.5 3
Standard   302 DataCores  0.5
END
```

Output Report File (report.txt)

Sample Output File

```
===== Galactic Logistics Hub Report =====
Vessel: Firefly [Type: LightFreighter] Cargo Items: 3 / 10
Total Effective Weight: 23.00 kg
```


Fuel Required (100 ly): 1150.00 units

Vessel: Goliath [Type: HeavyLifter] Cargo Items: 3 / 10

Total Effective Weight: 84.60 kg

Fuel Required (100 ly): 13190.00 units

Vessel: Phantom [Type: StealthRunner | CloakRating: 0.75] Cargo Items: 2 / 10

Total Effective Weight: 7.00 kg

Fuel Required (100 ly): 525.00 units

===== Cargo Load Log =====

[LOG] FoodSupplies loaded onto Firefly

[LOG] Uranium loaded onto Firefly

[LOG] Vaccines loaded onto Firefly

[LOG] MachinePartsA loaded onto Goliath

[LOG] MachinePartsB loaded onto Goliath

[LOG] Acids loaded onto Goliath

[LOG] BioSamples loaded onto Phantom

[LOG] DataCores loaded onto Phantom

Required Console Output Format

When `processLogistics()` is called, the console output matches the vessel-summary section of the report file above.

Important Note

The grader will compare your output character-by-character using a diff tool. Incorrect spacing, wrong decimal precision, a missing separator line, or a wrong type label will each incur deductions.

Effective weight calculation cross-check for the sample file:

Vessel	Cargo	Raw (kg)	Formula	Effective
Firefly	FoodSupplies	10.00	$\times 1.0$	10.00
	Uranium	5.00	$\times 1.2$	6.00
	Vaccines	2.00	$+5.0$	7.00
	Total			23.00
Goliath	MachinePartsA	40.00	$\times 1.0$	40.00
	MachinePartsB	35.00	$\times 1.0$	35.00
	Acids	8.00	$\times 1.2$	9.60
	Total			84.60
Phantom	BioSamples	1.50	$+5.0$	6.50
	DataCores	0.50	$\times 1.0$	0.50
	Total			7.00

Memory Safety Requirements

- **No memory leaks:** every `new` must have a matching `delete`.
- **No double deletes:** never delete a pointer more than once.
- **No dangling pointers:** do not use a pointer after deleting it.
- **No shallow copies:** all copy constructors and assignment operators must deep-copy owned memory.
- **Correct array deletion:** `new T[n]` must be freed with `delete[] ptr`, not `delete ptr`.
- **Destructor chaining:** Hub deletion $\rightarrow$ all Vessels deleted $\rightarrow$ all Cargo deleted.
- **Log list freed:** the full CargoLog linked list is freed in Hub's destructor.

File Handling Reference

This section summarises the C++ standard-library tools available for file I/O. How you use them to implement the required functions is left to you.

Important Note

The exact format of `fleet.txt` and `report.txt` is fixed by the provided sample files. The automated grader checks your output character-for-character. Do not add, remove, or reorder any fields.

Required Headers

```
1 #include <fstream>      // std::ifstream, std::ofstream
2 #include <sstream>      // std::stringstream, std::istringstream
3 #include <string>       // std::string, std::getline
4 #include <iomanip>      // std::fixed, std::setprecision, std::setw
```

Reading from a File — `std::ifstream`

`std::ifstream` opens a file for reading. Construct it with a filename; check `is_open()` before proceeding.

Operation	Notes
<code>ifstream f("name.txt");</code>	Opens the file at construction.
<code>f.is_open()</code>	Returns <code>true</code> if the file was opened successfully.
<code>std::getline(f, line)</code>	Reads one line (strips the newline) into <code>line</code> ; returns the stream, which is <code>false</code> at EOF.
<code>f » var</code>	Reads the next whitespace-delimited token into <code>var</code> .
<code>f.close()</code>	Closes the file explicitly (also called by destructor).

A typical read loop:

```
1 std::ifstream file("data.txt");
2 if (!file.is_open()) { /* handle error */ }
3 std::string line;
4 while (std::getline(file, line)) {
5     if (line.empty()) continue;
6     // process line
7 }
8 file.close();
```

Parsing a Line — `std::stringstream`

Wrap a `std::string` in a `std::stringstream` to extract whitespace-delimited tokens using the same `»` operator:

```
1 std::string line = "Token1 42 3.14";
2 std::stringstream ss(line);
3 std::string word; int n; double d;
4 ss >> word >> n >> d; // word="Token1", n=42, d=3.14
```

Subsequent `»` calls on the same `stringstream` continue from where the previous one left off, making it easy to parse fields of mixed types.

Writing to a File — `std::ofstream`

`std::ofstream` opens a file for writing (creates it if absent; truncates by default).

Operation	Notes
<code>ofstream out("report.txt");</code>	Opens / creates the file.
<code>ofstream out("log.txt", ios::app);</code>	Opens in append mode (does not truncate).
<code>out.is_open()</code>	Check before writing.
<code>out << value << std::endl;</code>	Writes a value followed by a newline and flush.
<code>out << std::fixed</code>	Enables fixed-point notation for floats.
<code>out << std::setprecision(n)</code>	Sets the number of decimal places.
<code>out.close()</code>	Flushes and closes the file.

Error Handling

Always verify that a file opened successfully before reading or writing. For read errors, printing to `std::cout` is acceptable; for write errors, prefer `std::cerr` so the message is not mixed with program output.

Important Note

Every dynamically allocated object (**new**) that is not stored in a container must be **deleted** to avoid memory leaks. If a **new** pointer is rejected by an add-method that reports capacity full, it is **your** responsibility to free it.

Quick Reference — Common Patterns

Goal	Idiom
Skip blank lines	<code>if (line.empty()) continue;</code>
Read until a sentinel	<code>while (getline(f, line) && line != "END")</code>
Check stream state	<code>if (!ss) / if (ss.fail())</code>
Write floating-point	<code>out << fixed << setprecision(2) << val;</code>
Append to existing file	<code>ofstream out(name, ios::app);</code>

File Structure

The skeleton package already contains every file listed below. **Do not rename, remove, or add files.** The automated grader expects this exact layout.

```
PA2/
|-- include/
|   |-- Entity.h
|   |-- Cargo.h
|   |-- StandardCargo.h
|   |-- HazardousCargo.h
|   |-- PerishableCargo.h
|   |-- Vessel.h
|   |-- LightFreighter.h
|   |-- HeavyLifter.h
|   |-- StealthRunner.h
|   |-- Inspector.h
|   |-- CargoLog.h
|   \-- Hub.h
|-- code/
|   |-- Entity.cpp
|   |-- Cargo.cpp
|   |-- StandardCargo.cpp
|   |-- HazardousCargo.cpp
|   |-- PerishableCargo.cpp
|   |-- Vessel.cpp
|   |-- LightFreighter.cpp
|   |-- HeavyLifter.cpp
|   |-- StealthRunner.cpp
|   |-- Inspector.cpp
|   |-- CargoLog.cpp
|   \-- Hub.cpp
|-- tests/
|   |-- testharness.h
|   \-- visible/
|       |-- t01_entity.cpp
|       |-- t02_cargo_types.cpp
|       |-- t03_vessel_base.cpp
|       |-- t04_vessel_ruleof3.cpp
|       |-- t05_derived_vessels.cpp
|       |-- t06_inspector.cpp
|       |-- t07_cargolog.cpp
|       |-- t08_hub_basic.cpp
|       |-- t09_hub_ruleof3.cpp
|       |-- t10_fileio.cpp
|       |-- t11_statics.cpp
|       \-- t12_polymorphism.cpp
|-- fleet.txt
\-- Makefile
```

Submission Requirements

You must submit exactly one **ZIP** file. The archive structure is strictly enforced by the autograder.

What to Include

Your ZIP file must contain **only** the following two folders:

```
include/  
code/
```

Nothing else must appear at the root level of the ZIP.

What NOT to Include

- `fleet.txt`
- `Makefile`
- Compiled executables (e.g., `.exe`)
- Object files (e.g., `.o`)
- Hidden system files (e.g., `.DS_Store`)
- Any additional folders

Correct ZIP Structure

When we unzip your file, the directory structure must look exactly like this:

```
include/  
    *.h  
code/  
    *.cpp
```

There must be **no outer parent folder**. The autograder will fail if an extra top-level folder exists.

Archive Naming

Name your file exactly:

```
RollNo_PA2.zip
```

Example:

```
27100345_PA2.zip
```

Important

If your ZIP does not extract into exactly `include/` and `code/` at the root level, your submission may receive zero marks due to autograder failure.

Grading Rubric

The assignment is graded out of **100 points**, split into **70% visible** (tested by the cases provided to you) and **30% hidden** (additional tests run by the grader after submission).

Testing Your Code

This section describes the automated test suite shipped with the skeleton project and explains how to use it to validate your implementation incrementally, one design layer at a time.

Quick Reference

Run the following command at any time to print a summary of all available targets directly in your terminal:

```
make help
```

Prerequisites

- **Compiler:** g++ supporting C++11 (g++ --std=c++11).
- **Build tool:** GNU make.
- **Working directory:** All commands must be run from the PA2/ root directory so that fleet.txt can be found by file I/O tests.

Directory Layout

```
PA2/
|-- include/          all header files (*.h)  -- do not modify
|-- code/             student implementation files (*.cpp)
|-- fleet.txt         sample fleet for file-I/O tests
|-- Makefile          student test runner
\-- tests/
    |-- testharness.h  lightweight test framework (read-only)
    \-- visible/       t01 - t12 (provided to you)
```

Grade Report (Default)

Running `make` (or the equivalent `make grade / make test-visible`) compiles and executes all 12 visible tests **one at a time**, printing a result line immediately after each one so you always see progress. At the end a weighted score out of 70 is shown.

```
make
```

Example output:

```
=====
PA2 - Galactic Logistics Hub | Grade Report
=====
Building t01 Abstraction (Entity)      [PASS]  5 /  5
Building t02 Cargo types (integration)  [PASS]   ---
Building t03 Encapsulation + Overloading [FAIL]  0 /  7
Building t04 Rule of Three - Vessel     [ERR ]  0 / 10
```



```

...
-----
VISIBLE TOTAL                                     5 / 70
=====
Hidden tests (30 pts) graded by TA.
=====

```

The three possible outcomes per line are:

- [PASS] – compiled and every check passed; full marks awarded.
- [FAIL] – compiled but at least one check failed; 0 for that layer.
- [ERR] – did not compile; 0 for that layer.

All 12 tests are always attempted regardless of earlier failures, so you see your full score in one run.

Debugging Layer by Layer

When a layer fails, run it individually to see the detailed check-by-check output:

```

make test-layer-01    # Entity (abstraction / ID counter)
make test-layer-02    # Cargo types
make test-layer-03    # Vessel base class
make test-layer-04    # Vessel Rule of Three
make test-layer-05    # Derived vessels
make test-layer-06    # Inspector (friend class)
make test-layer-07    # CargoLog (linked list node)
make test-layer-08    # Hub basic operations
make test-layer-09    # Hub Rule of Three
make test-layer-10    # File I/O (loadFleet / saveReport)
make test-layer-11    # Static member counters
make test-layer-12    # Polymorphism via base pointers

```

Each individual run prints one line per check and exits with code **0** on full pass or **1** on any failure:

```

[PASS] entity count starts at 0
[FAIL] copy ctor produces independent cargo (line 47)
      Expected: 2      Got: 0
----
SUMMARY: 11/12 passed

```

Mapping: Make Targets → Rubric Components

Target	Rubric Component	Points
test-layer-01	Abstraction (Entity)	5
test-layer-02	Cargo Inh. & Polymorphism (builds t05 foundation)	—
test-layer-03	Encapsulation + Constructor Overloading (Vessel)	7
test-layer-04	Rule of Three — Vessel (deep copy + self-assign)	10
test-layer-05	Inheritance & Polymorphism (vessels + clone)	15
test-layer-06	Friend Class (Inspector)	7
test-layer-07	Linked List (CargoLog)	4
test-layer-08	Function Overloading (<code>findVessel</code>)	2
test-layer-09	Rule of Three — Hub	5
test-layer-10	File Handling	8
test-layer-11	Static Members	7
test-layer-12	Runtime Polymorphism dispatch	(integration)
Visible Subtotal		70
Hidden tests		30
Total		100

Academic Integrity

- This is an **individual** assignment. All submitted work must be entirely your own.
- Zero tolerance for plagiarism. Similarity-detection tools will be applied to every submission.
- Sharing code, even as a “reference”, is prohibited and results in a **zero for all parties involved**.
- AI-generated code is **not permitted**. You are expected to understand and own every line you submit.
- You may discuss high-level concepts with peers or TAs, but all implementation must be your own.

Good Luck!

Start early. The hierarchy is designed so each class can be built and tested independently before integration. Recommended build order:

1. **Entity** → **Cargo** subtypes → **Vessel** subtypes
2. **Inspector** and **CargoLog**
3. **Hub** (file loading last)